DOCUMENTO OFICIAL DE AUDITORIA & ENGENHARIA DE SOFTWARE - TCC

OPUS: Auditoria Profunda, Correções de Lógica e Arquitetura Gamificada

Evolução do Duolingo de Programação Java para Apresentação Local e Dispositivos Móveis

PROJETO	AMBIENTE DE EXECUÇÃO	REFERÊNCIAS DE DESIGN
OPUS (Trabalho de Conclusão de Curso)	100% Localhost (XAMPP / PHP 8 / MySQL)	Duolingo, Mimo, SoloLearn e Exercism

1. Resumo Executivo & Escopo do Projeto de TCC

O **OPUS** é uma plataforma gamificada de ensino de programação Java concebida sob os princípios pedagógicos do *Microlearning* e do *Mobile Assisted Language Learning (MALL)*. O objetivo do projeto é substituir a leitura passiva e maçante de apostilas por uma experiência interativa, lúdica e prática, onde o aluno resolve desafios progressivos, avança em ligas competitivas, mantém ofensivas diárias (fogo/streak) e gerencia corações/vidas.

Premissa Fundamental da Apresentação do TCC: Todo o ecossistema deve operar **exclusivamente em ambiente local (offline/localhost)**. Não haverá deploy em provedores de nuvem pagos (AWS, Heroku, etc.). No dia da banca examinadora, a aplicação rodará no notebook do estudante e poderá ser acessada e testada simultaneamente em um smartphone conectado à mesma rede Wi-Fi local.

2. Auditoria Crítica do Código Atual: Erros de Lógica, Sintaxe e Segurança

Uma inspeção detalhada em todos os módulos do repositório (PHP, Stored Procedures MySQL, JavaScript e CSS) revelou falhas que comprometem o funcionamento durante a apresentação e que devem ser sanadas:

Gravidade	Localização	Diagnóstico da Falha	Impacto e Correção
Crítico	`back/api_chatbot.php` (linhas 260 e 308)	**Erro Fatal de Sintaxe (Parse Error):** Presença de barras invertidas corrompidas (`\$response`) inseridas por scripts de reparo desgovernados.	O PHP quebra com **HTTP 500** imediato ao abrir o chat. Além disso, a chamada externa tenta usar o modelo `openai/gpt-oss-20b` que **não existe**

Gravidade	Localização	Diagnóstico da Falha	Impacto e Correção
			na Groq. *Correção: Limpar escapes e ajustar modelo para* `llama-3.3-70b-versatile`.
Crítico	`test_api.php` (raiz) e scripts `fix_*.php`	**Backdoors e Mutadores Abertos:** `test_api.php` força `$_SESSION['user_id'] = 1` sem senha. Sete scripts `fix_*.php` alteram arquivos via web sem autenticação.	Permite que qualquer pessoa acesse como Administrador ou corrompa arquivos do backend. *Correção: Excluir imediatamente todos os scripts de teste e reparo da raiz.*
Alto	`back/licao_logic.php` (linhas 32-47)	**Bypass de Bloqueio de Lições:** O backend lê `$_GET['cap']` e `$_GET['licao']` sem checar se o aluno desbloqueou a lição em `progresso_usuario`.	Se o usuário alterar a URL para `licao.php?cap=1&licao=5` e acertar, a procedure do banco pula as lições anteriores e marca o capítulo como concluído. *Correção: Validar pré-requisitos antes de carregar o quiz.*
Alto	`opus.sql` (linha 734) e `front/pages/resultado.php`	**Quebra de Contrato na Pontuação:** A stored procedure `sp_processar_resultado_licao` exige acerto de 100% (3 de 3) para registrar progresso e XP.	Ao acertar 2 de 3, a tela diz "Muito bom! +25 XP", mas o banco **não grava nada**. Além disso, o aluno só perde vida se acertar 0 de 3. *Correção: Deduzir 1 vida por questão errada (como no Duolingo) e alinhar mensagens da UI ao retorno da procedure.*
Médio	`back/auth.php` (linha 123) e `redefinir_senha.php`	**Senhas em Plaintext:** Algumas senhas são salvas sem criptografia ou checadas com texto puro.	Violação clássica de segurança acadêmica apontada por bancas de TCC. *Correção: Padronizar com* `password_hash()` *e* `password_verify()`.

3. Arquitetura de Execução Java (100% Localhost para o TCC)

> **Por que Docker no Celular é Inviável e Nunca é Usado pelo Mercado:**
>
> O Android e o iOS utilizam sandboxes rígidas que impedem a execução de daemons com privilégios de sistema (cgroups/namespaces) sem que o celular tenha sido rooteado. Além disso, a JVM consumiria centenas de megabytes de RAM e esgotaria a bateria. Todas as ferramentas comerciais (**Mimo, SoloLearn, Codecademy, LeetCode**) operam com arquitetura **Thin-Client / Execução Remota**.

Diagnóstico do Ambiente Atual da Máquina:

Durante os testes no PowerShell, constatou-se que o comando `javac` e `java` retornam: "*O termo 'javac' não é reconhecido como nome de cmdlet...*". Ou seja: **não há compilador Java configurado no PATH do Windows atual**.

- **Ação Obrigatória:** Instalar o **OpenJDK 17 ou 21 LTS** (como o Eclipse Temurin) no computador da apresentação e adicionar a pasta `bin` à variável de ambiente `Path`.

Fluxo de Execução Cliente-Servidor na Apresentação:

Na apresentação, o smartphone do aluno e o notebook do apresentador estarão conectados no mesmo Wi-Fi. O smartphone envia o código via requisição HTTP para o notebook, que compila e executa o código Java de forma segura:

back/api_executar_java.php (Endpoint Seguro com Timeout)

```php
<?php
header('Content-Type: application/json; charset=utf-8');
session_start();

if (!isset($_SESSION['user_id'])) {
    http_response_code(401);
    echo json_encode(['sucesso' => false, 'erro' => 'Sessão expirada.']);
    exit();
}

$codigo = trim($_POST['codigo'] ?? '');
if (empty($codigo)) {
    echo json_encode(['sucesso' => false, 'erro' => 'Nenhum código enviado.']);
    exit();
}

// 1. Cria diretório temporário isolado por execução
$temp_dir = sys_get_temp_dir() . DIRECTORY_SEPARATOR . 'opus_' . bin2hex(random_bytes(6));
mkdir($temp_dir, 0777, true);
$java_file = $temp_dir . DIRECTORY_SEPARATOR . 'Main.java';
file_put_contents($java_file, $codigo);
```

```
// 2. Compilação com javac
$comp_out = [];
$comp_code = 0;
exec("javac \"$java_file\" 2>&1", $comp_out, $comp_code);

if ($comp_code !== 0) {
    array_map('unlink', glob("$temp_dir/*.*"));
    rmdir($temp_dir);
    echo json_encode([
        'sucesso' => false,
        'tipo'    => 'ERRO_COMPILACAO',
        'saida'   => implode("\n", $comp_out)
    ]);
    exit();
}

// 3. Execução segura com timeout de 3 segundos (evita loop infinito)
$escaped_dir = escapeshellarg($temp_dir);
$cmd = "powershell -Command \"\$p = Start-Process java -ArgumentList '-cp', $escaped_dir, 'Main' -NoN
exec($cmd, $out, $code);

$stdout = file_exists("$temp_dir/out.txt") ? file_get_contents("$temp_dir/out.txt") : '';
$stderr = file_exists("$temp_dir/err.txt") ? file_get_contents("$temp_dir/err.txt") : '';

// Limpeza de disco
array_map('unlink', glob("$temp_dir/*.*"));
rmdir($temp_dir);

if ($code === 124) {
    echo json_encode(['sucesso' => false, 'tipo' => 'TIMEOUT', 'erro' => 'Tempo limite de 3s excedido
    exit();
}

echo json_encode(['sucesso' => true, 'tipo' => 'SUCESSO', 'saida' => trim($stdout . "\n" . $stderr)])
?>
```

4. Estratégia de Migração para Celular (Mobile 100% Local)

Você não precisa programar um aplicativo em Kotlin ou Flutter do zero. A metodologia recomendada para o TCC baseia-se em **Capacitor (da equipe do Ionic)** ou **PWA (Progressive Web App)**:

Como demonstrar no celular sem gastar 1 centavo com servidor:

1. Conecte o notebook e o celular na **mesma rede Wi-Fi** (ou ative o ponto de acesso 4G do celular e conecte o notebook nele).
2. No notebook, execute `ipconfig` no prompt de comando e copie o endereço **IPv4** (exemplo: `192.168.1.105`).
3. No celular, abra o Chrome e digite: `http://192.168.1.105/OPUS/front/pages/licao.php`.
4. **Com o Capacitor:** O projeto web é empacotado em um aplicativo nativo Android real (`.apk`). O aluno instala o ícone no smartphone e desfruta de feedback de vibração tátil (Haptics) a cada acerto.

Acessibilidade de Código no Celular: Barra Rápida de Símbolos

Digitar código em celulares é cansativo devido à necessidade de alternar teclados para encontrar caracteres especiais. Para resolver isso, implemente uma barra de atalhos rápidos posicionada logo acima do teclado virtual:

```
; { } ( ) " " = System.out.println
```

5. Identidade Visual Cartoon, Animações e Sound Design (Vibe Duolingo)

A. Unificação da Identidade Visual (Design Tokens)

A investigação detectou que a tela de lição já possui botões 3D volumosos (com relevo que afunda ao clicar), mas o resto do sistema (Dashboard, Perfil, Amigos, Admin) está estilizado com uma estética *Dark/Cyberpunk* com a fonte espacial `Orbitron`.

- **Diretriz de Design:** Padronizar todas as telas com a identidade visual cartoon: fontes arredondadas (como `Nunito` ou `Poppins`), cores alegres do Duolingo (`#58cc02` verde, `#1cb0f6` azul, `#ff9600` laranja) e bordas suaves.

B. Mascotes Animados Dinâmicos (Lottie Files / Rive)

Atualmente, o mascote altera entre três arquivos estáticos PNG. Substituindo por animações vetoriais **Lottie (JSON animado)**, o mascote pisca, comemora com saltos, bate palmas e reage em tempo real com arquivos levíssimos de menos de 40 KB.

C. Sound Design Lúdico (Howler.js)

A gamificação psicológica do Duolingo apoia-se no feedback auditivo imediato. Integrando a biblioteca leve **Howler.js**, o OPUS passa a emitir:

- **Click Tátil:** Som de estalo suave ao tocar em qualquer opção ou bloco de código.
- **Chime Ascendente:** Acorde musical alegre e triunfal ao acertar uma questão.
- **Boing Cartoon Suave:** Som elástico e amigável ao errar (sem ser punitivo ou agressivo).
- **Fanfarra de Conclusão:** Trompetes com animação de chuva de confetes na tela final de resultados.

6. Novos Formatos de Exercícios de Programação (Mimo, SoloLearn e Exercism)

Formato	Dinâmica & Interação	Vantagem no Mobile e TCC
Parson's Problems (Ordenação de Blocos)	O aluno recebe 4 ou 5 linhas de código Java embaralhadas e deve arrastá-las com o dedo para a ordem de execução correta.	**Perfeito para celular:** Zero digitação no teclado virtual; interação 100% tátil e visual.
Token Bank (Banco de Chips)	O código Java possui lacunas `_____`. Abaixo, o aluno clica em chips (ex: `[public]`, `[class]`, `[int]`) que sobem e preenchem o espaço.	Fixa a sintaxe e palavras-chave de Java de maneira dinâmica e instantânea.
Spot the Bug (Caça ao Erro)	Apresenta um trecho de código com um erro de sintaxe clássico (ex: ausência de `;` ou tipo incompatível). O aluno toca na linha incorreta.	Ensina habilidades reais de depuração (debugging) valorizadas pelo mercado de trabalho.
Boss Fight (Desafio Prático Final)	Última lição de cada capítulo: o aluno escreve uma rotina completa no editor e clica em "Executar Código". O backend compila via `javac` e valida os testes.	Comprova formalmente para a banca que o projeto não é apenas um quiz conceitual, mas uma IDE de ensino completa.

7. Plano de Ação e Roteiro de Implementação para a Banca

1. **Fase 1 (Saneamento Imediato):** Excluir os scripts temporários (`test_api.php` e `fix_*.php`) e corrigir a sintaxe de `api_chatbot.php`.
2. **Fase 2 (Configuração do Ambiente Java):** Instalar o OpenJDK 17/21 LTS no notebook e validar `javac -version` no terminal.
3. **Fase 3 (Endpoint de Execução Local):** Implantar o arquivo `back/api_executar_java.php` com proteção de timeout.
4. **Fase 4 (Interface & Gamificação):** Inserir a biblioteca de som **Howler.js** na lição e adicionar a barra de atalhos rápidos de símbolos de código.
5. **Fase 5 (Demonstração Prática):** Conectar o celular no Wi-Fi do notebook e demonstrar a resolução de lições e execução de código Java em tempo real para a banca examinadora.
